

Syntax vs Semantics

↳ structure / Grammar / form

↳ meaning / behaviour

↳ is this written correctly?

↳ what does this do?

→ I eat apple ✓ (synactically)

eat 9 apple x

→ int x = 10/0;

Syntax ✓

Semantics → runtime error

→ int x = "hello";

Syntax ✓

Semantic x → type mismatch

* Scanner checks syntax at token level

* parser checks syntax at Grammar level

* Interpreter / Compiler handles semantics

Language Design

Tree-walk interpreter

Bytecode vm / compiler

1. Typing System

→ Dynamic typing (like Python, JS)

→ Static typing (like Java, C++)

2. Execution model → Do you want?

→ Interpreted

→ Compiled to bytecode

→ Compiled to native

3. Syntax style → c-style?

var x = 10;

if (x > 5) { → or → indentation based like Python.

print x;

}

memory model

→ Garbage collected?

→ Manual?

→ Reference counting?

In java-based interpreter → JVM handles memory for us.

* pipeline → Source code → Scanner (lexer)

Interpreters ← Semantic Analysis ← AST ← Parser ← Tokens
or
Code Generator

Grammars:

→ Regular Grammar → used in:

* DFA / NFA

* Token recognition

Ex: Number → digit +

→ Context Free Grammar (CFG) → used in:

* Parsing

* Expression Structure

Ex: expression → expression "+" expression

* Regular Grammar is not enough bcz regular languages (DFA) can't handle nested parentheses, bcz matching parentheses requires stack memory, and DFA has no stack.
→ Parser uses recursion (stack).

Ex:-expression $\rightarrow$ termterm $\rightarrow$ factor $(("+" | "-") \text{factor})^*$ factor $\rightarrow$ unary $(("*" | "/") \text{unary})^*$ unary $\rightarrow$ $("-" \text{unary}) | \text{primary}$ primary $\rightarrow$ NUMBER $| "(" \text{expression} ")"$ $\rightarrow$ operators have levels:Level 1: highest priority $\rightarrow$ $()$ Level 2: $\rightarrow$ unary - ~~not~~Level 3: $\rightarrow$ $*$, $/$ Level 4: $\rightarrow$ $+$, $-$ $\rightarrow$ An expression is made of terms $\Rightarrow$ expression $\rightarrow$ termA term is multiplication parts combined by $+$ or $-$ $\Rightarrow$ term $\rightarrow$ factor $(("+" | "-") \text{factor})^*$

• first read a factor

• Then allow $+$ or $-$ factor

• many times

• A factor handles $*$ and $/$ $\Rightarrow$ factor $\rightarrow$ unary $(("*" | "/") \text{unary})^*$ A unary handles: -5 $\Rightarrow$ unary $\rightarrow$ $"-" \text{unary} | \text{primary}$

A primary is most basic things:

 $\Rightarrow$ primary $\rightarrow$ NUMBER $| "(" \text{expression} ")"$ $\rightarrow$ primary $\rightarrow "(" \text{expression} ")"$, it means if you see (something) then parse "something" as a full expression.

It allows nesting

Ex: $(3 + (4 * 5))$

स्पष्ट मूल्य-वचित मूल्य

1. Left Associativity $\rightarrow$ Group from left

$$a - b - c = (a - b) - c$$

operators: $+$, $-$, $*$, $/$

2. Right Associativity $\rightarrow$ Group from right

$$a = b = c \rightarrow a = (b = c)$$

operators: Assignment, Exponentiation (in some languages)

$\rightarrow$ term $\rightarrow$ factor $(("+" | "-") \text{ factor})^*$

This pattern automatically produces LEFT ASSOCI. bcoz, it parses like:

1. read first factor

2. then repeatedly attach to right

So, for, $8 - 3 - 2$

$$\begin{array}{c} \\ \\ \\ \\ \end{array} = (8 - 3) - 2$$

$\Rightarrow$ make subtraction Right asso. by

changing rule. how will you do this?

term $\rightarrow$ factor $(("+" | "-") \text{ term})$

or even more clearly

expression $\rightarrow$ factor | factor $"-"$ expression

→ Grammar to code (roughly idea)

if we have rule :

primary $\rightarrow$ NUMBER | "(" expression ")"

then, parser method will look like :

```
Expr primary() {
    if (match(NUMBER)) {
        }
```

```
    if (match(LEFT-PAREN)) {
        }
```

factor $\rightarrow$ unary ("(" * " | "/") unary) *

```
Expr factor() {
    Expr expr = unary();
    while (match(STAR, SLASH)) {
        Token operator = previous();
        Expr right = unary();
        expr = new Binary(expr, operator, right);
    }
    return expr;
}
```

→ Each rule = one function

→ Grammar repetition (*)

while loop

→ Grammar recursion

method calling itself

→ Grammar OR (|)

if/else branches

If input is : 123.45, what scanner should do?

- Treat it as characters
- Convert to double
- recognise full number token.

→ Scanner does not understand grammar
it only understands → characters
→ patterns
→ Tokens

It answers: what is this piece of text?

80. $\text{input} = 123.45 + 6$

[NUMBER(123.45), PLVS, NUMBER(6), EOF]

Scanner = Airport security check

It identifies $\rightarrow$ This is a passport

↳ This is a message

→ This is metal object

it does not care where

passenger is going and what will he do with this items.

→ Scanner internally behaves like DFA.

Ex: reading a number

1 → do grt

2 → digit

→ decimal point

$h \rightarrow \text{class}$

So, Scanner transitions through states:
START $\rightarrow$ digit $\rightarrow$ digit $\rightarrow$ decimal point or digit
digit

$\hookrightarrow$ That is deterministic state machine.

$\rightarrow$ Step 1: Define Tokens Types

First we define all tokens our language supports.

- LEFT_PAREN
- RIGHT_PAREN
- PLUS
- MINUS
- STAR
- SLASH
- NUMBER
- EOF

$\rightarrow$ Java Enum:

```
enum TokenType {
    LEFT_PAREN, RIGHT_PAREN, PLUS, MINUS, STAR,
    SLASH, NUMBER, EOF.
}
```

$\rightarrow$ why enum: because

- $\rightarrow$ fixed set of tokens
- $\rightarrow$ Type-safe
- $\rightarrow$ Efficient
- $\rightarrow$ cleaner than strings

Step 2: Token Class

Each token needs:

Type

Lexeme (actual text)

Literal value (for numbers)

Line number (for error reporting)

```
class Token {
```

```
    final TokenType type;
```

```
    final String lexeme;
```

```
    final Object literal;
```

```
    final int line;
```

```
    Token (---) {
```

```
        ---
```

```
    } // constructor
```

```
    public String toString() {
```

```
        return type + " " + lexeme + " " + literal;
```

```
    }
```

```
}
```

input = 123.45

lexeme = "123.45" → original text

literal = 123.45 (double) → parsed value

Step 3:

PRAGATI
Page: _____
Date: _____

→ Scanner Skeleton:

```
class Scanner {  
    private final String source;  
    private final List<Token> tokens = new ArrayList<>();  
    private int start = 0;  
    private int current = 0;  
    private int line = 1;  
  
    Scanner(String source) {  
        this.source = source;  
    }  
  
    List<Token> scanTokens() {  
        while (!isAtEnd()) {  
            start = current;  
            scanToken();  
        }  
        tokens.add(new Token(Token.Type.EOF, "", null, line));  
        return tokens;  
    }  
  
    private boolean isAtEnd() {  
        return current == source.length();  
    }  
  
    // private void scanToken() {  
        // logic is on next page.  
    }  
}
```


→ Scanner needs?

Computer reads: $12 + 5$, just characters
but parser does not want characters

Parser wants: NUMBER, PLUS, NUMBER

→ So, Scanner groups into meaningful chunks

→ to correctly extract tokens, scanner needs:

1. Entire source thing
2. where current token started
 - ↳ bcoz tokens can be multiple characters long.
3. where we are currently reading
4. line number

So, logic is:

while not end:

mark beginning of token

read characters

decide token type

add token.

→ can use regex to split tokens, but

↳ Hard to maintain

↳ Hard to extend

↳ Poor control

↳ Hard error reporting

↳ Doesn't teach parsing theory.

→ EOF means ending.

→ Regex: Regular - Expression

↳ A formal way to describe patterns in text

Ex:- $[0-9]^+$ means!

→ any digit from 0 to 9
 This matches: → one or more times
 123, 4.45, 5, 99

→ $[0-9]^+(\backslash\.[0-9]^+)?$

→ Digits

→ optional Decimal part

→ $[a-zA-Z_][a-zA-Z0-9_]*$

→ Start with letter or underscore

→ followed by letters or digits

↳ If a pattern can be recognised by DFA, it can be written using regex.

→ scanToken() :- When scanner reads one character, it must ask:

1. Is it a single character token?

→ (

→)

→ +

→ -

→ *

→ /

if yes → add token immediately

2. Is it white space?

↳ yes → ignore it

3. Is it newline?

↳ increase line count

4. Is it digit?

↳ Then it is beginning of number

↳ continue consuming digits

↳ possibly consume decimal

↳ add NUMBER token.

5. otherwise → error

```
private void ScanToken () {
```

```
    char c = advance ();
```

```
    switch (c) {
```

```
        case '(':
```

```
            addToken (TokenType, LEFT_PAREN);
```

```
            break;
```

```
        case ')':
```

```
            addToken (TokenType, RIGHT_PAREN);
```

```
        :
```

```
        case ' ':
```

```
        case '\r':
```

```
        case '\t':
```

```
            break;
```

```
        case '\n':
```

```
            line++;
```

```
            break;
```

```
        default:
```

```
            if (isDigit(c)) {
```

```
                number (c);
```

```
            } else {
```

```
                // consume special character
```


→ State transitions looks like:

START → digit → digit → digit → digit
we remain in digit state as long as next character is digit.

↳ only when next character is not digit, we stop.

→ ← Parser → ←

* why can't parser just calculate result directly? → why build AST?

→ input: $3 + 4 * 5$

if parser immediately calculates result it would compute:

$$4 * 5 = 20$$

$$3 + 20 = 23 \quad \text{return } 23$$

what if we want variables?

input: $x + 4 * y$

parser can't compute anything, becoz

→ it doesn't know values of x and y

→ values are known at runtime.

So, parser must build structure first.

what if we want to print AST?

print $(3+4)$;

→ we might want:

- Pretty printer
- Debug tree viewer
- Static Analysis
- Optimization.

if we directly evaluate ~~एक~~ मूल्य-उचित मूल्य
Tree is lost.

what if we add Feature later?

Suppose we later add:

if $(3+4)5$ {-----}

→ parser must:

- represent condition
- Represent block
- Represent statement

→ parser and execution are two different phases:

* parser's job: understand structure.

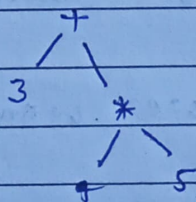
* Interpreter's job: Execute structure

→ Tree (AST) captures:

- * precedence
- * Associativity
- * Nesting
- * Parenthesis
- * operator grouping

Ex:-

$3 + 4 * 5$



→ AST does not contain:

- white space
- Parenthesis (usually)
- Extra formatting

becz ^{स्पष्ट मूल्य-उचित मूल्य} AST represents meaning, not text formatting.

→ From our Grammar we need these expression types:

1. Literal (for numbers)
2. Unary (for $-x$)
3. Binary (for $x+y$, $x*y$)
4. Grouping (for (expression))

→ Literal: Represents a value directly

Ex:- 3

Tree node: Literal(3) ↘

Leaf node, No childrens.

→ Unary: Ex:- -5

Tree: Unary($-$, 5) ↘ structure

- operator
- right expression

→ Binary: Ex:- $3+4$

Tree: Binary(3, $+$, 4) ↘ structure

- left expression
- operator
- right expression

→ Grouping: Ex:- $(3+4)$

Tree: Grouping(Binary(3, $+$, 4))

(why need Grouping Node? ↘

beoz, parenthesis affect structure during parsing, even though parenthesis disappear later in evaluation, they matter while building tree.

AST Structure:

we create an abstract base class:

Expr

Then subclasses:

Literal

Unary

Binary

Grouping

why abstract?

bcz we never create plain Expr, only specific types. (Literal, Binary etc)

if parser reads: $3 + 4$

It will create:

```
new Binary (
    new Literal(3),
    PLUS token,
    new Literal(4)
)
```

Parser's work

Scanner gives:

NUMBER(3)

PLUS

STAR

NUMBER(5)

EOF

To convert these flat tokens into hierarchical AST.

Flat list $\rightarrow$ Tree Structure

input: List <Tokens>

Output: Expr (AST root)

parser will return only Expr, which is the tree of whole expression.

- Recurring Descent : meaning of RD :
- ↳ each grammar rule = one java method
 - ↳ Grammar recursion = method recursion
 - ↳ Grammar repetition (*) = loop

Ex:- expression → Term

↓ becomes

```
Expr expression() {
    return term();
}
```

→ Parser Skeleton:

```
public class Parser {
```

```
    private final List<Token> tokens;
```

```
    private int current = 0;
```

```
    public Parser (List<Token> tokens) {
        this.tokens = tokens;
```

```
    }
```

// expression → term

```
private public Expr expression() {
    return term();
}
```

// term → factor (("+" | "-") factor) *

```
private Expr term() {
    // implementation
}
```

// factor → unary (("*" | "/") unary) *

```
private Expr factor() {
    // implementation
}
```


1) unary $\rightarrow$ "-" unary | primary
 private expr unary C.S.

2) primary $\rightarrow$ NUMBER | "(" expression ")"
 private expr primary C.S.

{

}

→ ← Interpreter → ←

Abhi pipeline hai ↴

Source → Tokens → AST

↓ Interpreter

AST → Result

→ Role of Interpreter:

Interpreter will do Tree walk and!

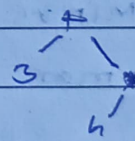
- Literal → naive return
- unary → ~~return~~ evaluate the operand
- binary → evaluate left and right
- Grouping → will evaluate inner.

→ Interpreter will be recursive since, AST is recursive.

→ Evaluation Strategy:

- ↳ Evaluation will happen in post order traversal, means
 - ↳ first evaluate children
 - ↳ Then parent

Ex: $3 + 4 * 5$



1. Evaluate 3

2. Evaluate 4

3. Evaluate 5

4. $4 * 5 = 20$, apply *

5. $3 + 20 = 23$, apply +

→ Interpreter code Structure:

evaluate (exp):

if literal → return value

if Grouping → evaluate (inner)

if unary → evaluate right then apply operator

if Binary → evaluate left and evaluate right then
apply operator.

→ return type is object, but in future
there will be strings, booleans and
nil. will come.